

Optimization

4.3.3 Constraints in practice: penalize, reject or compare

José Firmino Aguilar Madeira

One problem, five treatments

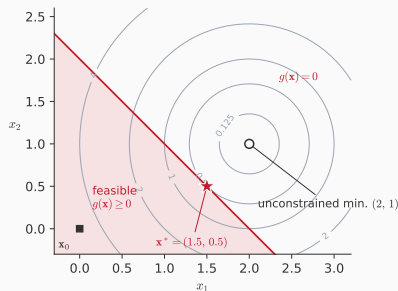
$$\min f(\mathbf{x}) = (x_1 - 2)^2 + (x_2 - 1)^2 \quad \text{s.t.} \quad g(\mathbf{x}) = 2 - x_1 - x_2 \geq 0$$

- $\mathbf{x}^* = (1.5, 0.5)$, $f^* = 0.5$, $\lambda^* = 1$ (KKT, 4.2); the unconstrained minimum $(2, 1)$ is **infeasible**.
- Violation: $v(\mathbf{x}) = -\langle g(\mathbf{x}) \rangle = \max(0, -g(\mathbf{x}))$, as in 4.3.1.

The **same** Hooke–Jeeves (3.2.5), from $\mathbf{x}^{(0)} = (0, 0)$, with the constraint communicated in five ways:

A ignore • **B** quadratic penalty • **C** exact penalty • **D** extreme barrier • **E** feasibility rule

The problem is the same; only the way we **communicate the constraints to the algorithm** changes.



Does the algorithm know there is a constraint?

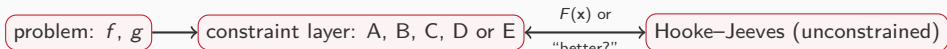
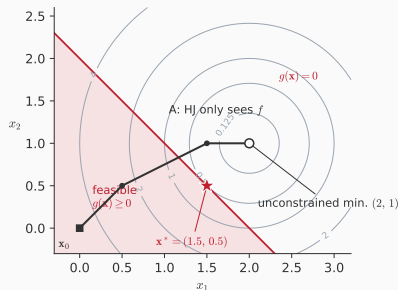
A — ignore. HJ receives only $F = f$ and returns

$\mathbf{x} = (2, 1)$, $g = -1$: infeasible ($n_F = 96$).

- HJ (and all of Ch. 3) minimizes **the number we give it**; it does not know what g is.
- The constraint only exists for it if it is **in that number** — or **in the comparisons** it makes.
- The algorithm stays **unconstrained**; feasibility is an **outer layer**.

n_F : calls to the function (or pair) that HJ receives;

$a = 2$, $P_0 = 0.5$, $T = 10^{-6}$ throughout the deck.



Can we charge for the violation?

B — quadratic penalty, with **fixed** R :

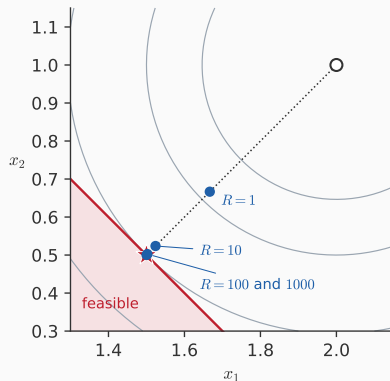
$$F_R(\mathbf{x}) = f(\mathbf{x}) + R \langle g(\mathbf{x}) \rangle^2 = f(\mathbf{x}) + R v(\mathbf{x})^2.$$

On the infeasible side, $\nabla F_R = \mathbf{0}$ gives

$$\mathbf{x}_R = (2, 1) - R v_R (1, 1), \quad v_R = \frac{1}{1 + 2R}.$$

- $v_R > 0$ for **every** finite R : always (slightly) **infeasible**.
- $\mathbf{x}_R \rightarrow \mathbf{x}^*$ **from outside**: it is the penalty of 4.3.1, but with fixed R and a single minimization.
- HJ finds these $\mathbf{x}_R$.

Paying for the violation **does not forbid it**.



How large should R be?

R	$\mathbf{x}_R$ (HJ)	$v(\mathbf{x}_R) = 1/(1 + 2R)$	$\ \mathbf{x}_R - \mathbf{x}^*\ $	n_F
1	(1.667, 0.667)	0.333	0.236	253
10	(1.524, 0.524)	0.048	0.034	244
100	(1.502, 0.502)	0.0050	0.0035	311
1000	(1.500, 0.500)	0.0005	0.00035	644

Small R

Cheap, but **visibly infeasible**: with $R = 1$, $v = 1/3$.

Large R

Almost feasible, but F_R has a **narrow valley** ($\kappa \sim R$, 4.3.1): HJ “crawls” — 644 calls.

The “right” R depends on the **scale** of g (4.3.1). The cure of 4.3.1 is an **increasing** R , with warm start; with fixed R , one chooses between infeasible and ill-conditioned.

Can we make the penalty exact?

C — exact penalty (ℓ_1): one pays v , not v^2 :

$$F_R = f + Rv = f + R|\langle g \rangle|.$$

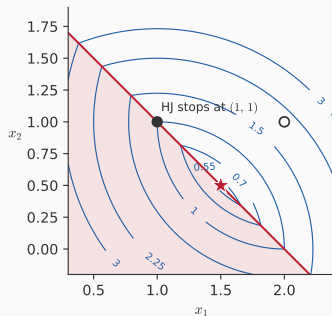
A local result

If $\mathbf{x}^*$ is a **strict** local minimizer and satisfies the KKT conditions with multipliers λ_j^* , then $\mathbf{x}^*$ is a **local** minimizer of $f + R \sum_j |\langle g_j \rangle|$ for every $R > \max_j \lambda_j^*$.

(Nocedal and Wright, 2006, Thm. 17.3)

Here $\lambda^* = 1$. HJ gives:

- $R = 0.5 < \lambda^*$: $(1.75, 0.75)$, infeasible;
- $R = 1 = \lambda^*$ (outside the theorem): here it still gives $\mathbf{x}^* = (1.5, 0.5)$;
- $R = 2 > \lambda^*$: the minimizer of F_R **is** $\mathbf{x}^*$... but HJ **stops at** $(1, 1)$.



Level sets of F_2 : a **kink** at $g = 0$.

The price: an upper bound on λ^* (unknown), and F_R is **not differentiable** where $\mathbf{x}^*$ lies. A **local** result, under assumptions — not universal.

Why did Hooke–Jeeves stop at $(1, 1)$?

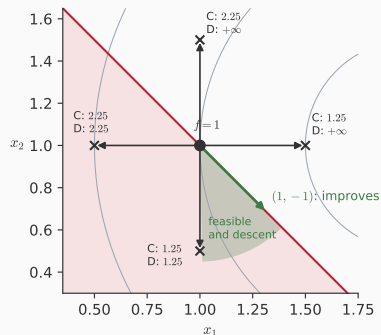
At $(1, 1)$, $f = 1$. The trials $+Pe_1$, $+Pe_2$ leave X (C pays Rv , D gives $+\infty$); $-Pe_1$, $-Pe_2$ stay in X , but f increases.

For **any** $P > 0$, no $\pm e_j$ improves: in D, the directions that descend **and** stay feasible are the green wedge; in C, those that descend F_2 are the quadrant $d_1 > 0$, $d_2 < 0$. HJ halves P down to T and stops at a **non-optimal** point.

Fix: change the directions, not the method

The same HJ in y , with $x = Qy$, $Q = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$: directions **along** and **normal to** the boundary. C ($R = 2$), D and E reach x^* ($n_F = 208$ in all three).

With linear constraints, the directions must follow the nearly active constraints (Lewis and Torczon, 2000).



What if we simply refuse infeasible points?

D — extreme barrier:

$$F(\mathbf{x}) = \begin{cases} f(\mathbf{x}), & g(\mathbf{x}) \geq 0, \\ +\infty, & g(\mathbf{x}) < 0 \text{ (} f \text{ is not evaluated).} \end{cases}$$

- No R , no scales: an infeasible point **always loses**; f is **never** evaluated outside X .
- **Feasible** iterates, provided $\mathbf{x}^{(0)} \in X$: from $(3, 3)$ all is $+\infty$ and HJ stays put.
- From $(0, 0)$: stops at $(1, 1)$; $n_F = 91$, 45 of them outside X (f not evaluated).
- Not the log barrier (4.3.2): no wall, just a “no”.

```
function y = ExtremeBarrier(f, g, x)
if any(g(x) < 0)
    y = Inf;           % f is not evaluated
else
    y = f(x);
end
end
```

This is what GLODS and DMS do

GLODS (5.1), MultiGLODS and DMS (6.5):
 $f = +\infty$ outside X : the objective function
is not evaluated at infeasible points.

What if there is not even a $g(x)$?

Explicit constraint

We know g : **whether** and **by how much** it is violated. There is a v : B, C, E are possible. Constraint taxonomy for simulation-based optimization: Le Digabel and Wild (2023).

```
function y = simulador(x)
if x(1) + x(2) > 2, y = NaN;
else, y = (x(1) - 2)^2 + (x(2) - 1)^2; end
end
F = @(x) ExtremeBarrier(@simulador, [], x);
x = HookeJeeves(F, [0 0], 2, 0.5, 1e-6);
```

Hidden constraint

The simulator **fails** (NaN) without saying why or by how much. No v : of the strategies here, **only D** needs no violation measure.

```
% black box
% "does not converge"

% NaN -> Inf
```

Result: **the same run** as D — $(1, 1)$, $n_F = 91$ — without ever knowing g . The 45 failures also **cost**: the simulator ran until it failed.

Do we need to add objective and violation?

E — feasibility rule. HJ only needs to know **which of two points is better**. Compare the pair (v, f) — no new function:

1. feasible **beats** infeasible;
2. between feasible points, smaller f wins;
3. between infeasible points, smaller v wins.

This is Deb's rule (2000), used in NSGA-II (6.6).

- From $(0, 0)$: same base points and same $(1, 1)$ as D ($n_F = 91$).
- From $(3, 3)$: D stays put; E reduces v and reaches $(1, 1)$ ($n_F = 96$).
- Needs v : explicit constraints only (f is needed only when the v tie).

MATLAB

```
FE = @(x) [v(x), f(x)];           % the pair (v, f)
menor = @(a, b) a(1) < b(1) || ...
        (a(1) == b(1) && a(2) < b(2));
x = HookeJeeves(FE, [3 3], 2, 0.5, 1e-6, ...
                [], false, menor);
```

Python

```
FE = lambda x: (v(x), f(x))       # the pair (v, f)
menor = lambda a, b: a[0] < b[0] or \
        (a[0] == b[0] and a[1] < b[1])
r = hooke_jeeves(FE, [3, 3], 2, 0.5, 1e-6,
                 menor=menor)
```

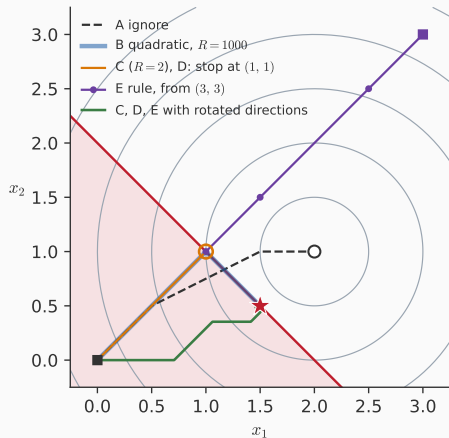
In the course HJ, `menor` ("less") is optional (by default, $a < b$).

The same problem, five treatments

Treatment	What HJ receives	Result (n_F)	Feasible?	Lesson
A ignore	f	(2, 1) (96)	no	the algorithm does not see g
B quadratic	$f + R v^2$, $R = 1000$	(1.500, 0.500), $v = 5 \cdot 10^{-4}$ (644)	no (almost)	approaches from outside; large R is costly
C exact	$f + R v$, $R = 2$	(1, 1) (91)	yes	$\mathbf{x}^*$ minimizes F_R , but there is a kink
D extreme barrier	f , or $+\infty$ without evaluating f	(1, 1) (91)	yes	never leaves X ; needs $\mathbf{x}^{(0)} \in X$
E rule	comparisons of (v, f)	(1, 1) (91); from (3, 3): (1, 1) (96)	yes	no sum; starts from outside
C, D, E rotated	the same; directions $Q\mathbf{e}_j$	$\mathbf{x}^* = (1.5, 0.5)$ (208)	yes	the directions matter

B and C **change the function**; D **refuses** points; E **changes the comparison**. The algorithm is always the same.

The paths



- In X , all of them see **the same** f : from (0, 0), the paths coincide up to the boundary.
- There they split: A crosses it; B slides along it from outside; C and D stop at (1, 1).
- E, from (3, 3), first **reduces the violation**; once in X , it compares like D.
- With the rotated directions, the path follows the boundary up to $\mathbf{x}^*$.

How to choose

explicit and cheap $g \rightarrow$ C or E (or B with increasing R , 4.3.1).

expensive f , or undefined outside $X \rightarrow$ D. Hidden constraint (no v) $\rightarrow$ D.

On the computer: the five treatments in MATLAB and Python

MATLAB

```
f = @(x) (x(1) - 2)^2 + (x(2) - 1)^2;
g = @(x) 2 - x(1) - x(2);           % g(x) >= 0
v = @(x) max(0, -g(x));             % violation
FA = f;                             % A: ignore
FB = @(x) f(x) + R*v(x)^2;          % B: quadratic
FC = @(x) f(x) + R*v(x);           % C: exact
FD = @(x) ExtremeBarrier(f, g, x);  % D: barrier
FE = @(x) [v(x), f(x)];             % E: the pair
menor = @(a, b) a(1) < b(1) || ...
    (a(1) == b(1) && a(2) < b(2));

x = HookeJeeves(FD, [0 0], 2, 0.5, 1e-6);
x = HookeJeeves(FE, [0 0], 2, 0.5, 1e-6, ...
    [], false, menor);
```

Python

```
f = lambda x: (x[0] - 2)**2 + (x[1] - 1)**2
g = lambda x: 2 - x[0] - x[1]        # g(x) >= 0
v = lambda x: max(0.0, -g(x))        # violation
FA = f                               # A: ignore
FB = lambda x: f(x) + R * v(x)**2    # B: quadratic
FC = lambda x: f(x) + R * v(x)       # C: exact
FD = lambda x: extreme_barrier(f, g, x) # D
FE = lambda x: (v(x), f(x))          # E: the pair
menor = lambda a, b: a[0] < b[0] or \
    (a[0] == b[0] and a[1] < b[1])

r = hooke_jeeves(FD, [0, 0], 2, 0.5, 1e-6)
r = hooke_jeeves(FE, [0, 0], 2, 0.5, 1e-6,
    menor=menor)
```

The full script (table, v_R , hidden constraint, start at (3, 3), stall, rotated directions) is in the course companion code, with Colab.

KEY TAKEAWAYS — 4.3.3

1. An unconstrained method sees only **numbers** (or comparisons): constraints enter via an outer layer.
2. **Penalizing** (B, C) changes the function: quadratic approaches from outside, ill-conditioned; exact makes $\mathbf{x}^*$ a local minimizer for finite $R > \lambda^*$ (under assumptions), but the algorithm may miss it.
3. **Refusing** (D) keeps the iterates feasible if one starts from a feasible point; of the strategies here, it is the only one needing no violation measure.
4. **Comparing** (E) needs no R and no sums, and starts from infeasible points.
5. With nonsmooth functions or barriers, the method's **directions** decide whether $\mathbf{x}^*$ is reached.

Bridge: GLODS (5.1), DMS (6.5): extreme barrier; GAs (5.3): penalty/rejection; NSGA-II (6.6): feasibility rule.

Beyond the course: the progressive barrier

Between D and E: it accepts infeasible points whose violation is below a threshold that **decreases** along the iterations, and refuses the others, like D (Audet and Dennis, 2009; MADS/NOMAD).

Turbine layout of a wind farm (IDMEC, M4WIND project)

Decision the coordinates (x_i, y_i) of each of the N turbines.

Objective the annual energy produced, net of wake losses.

Constraints minimum distance between turbines — $g_{pq} = \|(x_p, y_p) - (x_q, y_q)\| - d_{\min} \geq 0$, one per pair: $J = N(N - 1)/2$ inequalities ($N = 30$: 435); site boundary and exclusion zones (geometric constraints on each turbine).

What changes vs. the textbook f is multimodal and has no reliable derivatives; KKT by hand (2^J cases) is unthinkable. The constraints typically enter by **penalty** (4.3.1), by extreme barrier/repair or by the feasibility rule (4.3.3, 6.6), within a method from Ch. 5.

The information we have: values of f (expensive) and *geometric* constraints that are cheap to check. This asymmetry — expensive f , cheap g — decides how constraints are handled in a real problem (with D, f is not even evaluated at an infeasible point).

Summary table — Chapter 4

What information do we have about f and the constraints? What does it cost to obtain? What is guaranteed?

Method	Information	How it is solved / cost	Guarantee	Main limitation
Lagrange (4.1)	$\nabla f, \nabla h_\ell$ analytical	by hand: solve $n + K$ equations	necessary under independence of the ∇h_ℓ	only candidates; nonlinear system
KKT (4.2)	$\nabla f, \nabla g_j, \nabla h_\ell$	by hand: up to 2^J cases (KKT is not an algorithm)	necessary under LICQ; sufficient if convex	exponential enumeration by hand
Exterior penalty (4.3.1)	f, g_j, h_ℓ (+ gradients, if used internally)	$\sum_t n_f$ of the inner minimizations	$\mathbf{x}_R \rightarrow \mathbf{x}^*$ as $R \rightarrow \infty$ (under assumptions [†])	$\kappa \sim R$; slightly infeasible solution
Log barrier (4.3.2)	f, g_j ; interior point	idem, + Phase I	$\mathbf{x}_R \rightarrow \mathbf{x}^*$ as $R \rightarrow 0$ (under assumptions [†]); $\lambda_j = R/g_j$	only inequalities; $\kappa \sim 1/R$
In practice (4.3.3)	f, g_j or just “failed”	one minimization	C: local exactness if $R > \lambda^*$ (under assumptions); D, E: depends on the method	the directions

[†] f, g_j (h_ℓ) continuous; $\mathbf{x}_R$ global minimizers of P , with $(\mathbf{x}_R)$ bounded; $\mathbf{x}^*$ unique (without uniqueness: every limit point is a global minimum); for the barrier, also $X = \text{closure of the interior}$. For study. In class: the next two pages.

KEY TAKEAWAYS — Chapter 4

1. Constraints change the **directions** in which we can improve f .
2. The **multipliers** tell which constraints “count” and what they are worth.
3. **CHARACTERIZE** (4.1–4.2): Lagrange ($h = 0$: $\nabla f + \sum \lambda_\ell \nabla h_\ell = \mathbf{0}$) and KKT ($g \geq 0$: $\lambda \geq 0$, $ug = 0$) use derivatives $\rightarrow$ first-order conditions $\rightarrow$ candidates; global if convex.
4. **COMPUTE** (4.3): exterior ($R \uparrow \infty$, $\mathbf{x}_R \rightarrow \mathbf{x}^*$ from outside) and interior ($R \downarrow 0$, from inside) use Ch. 3 repeatedly $\rightarrow$ cost = sum of the inner minimizations $\rightarrow$ convergence under assumptions.
5. Turning the constrained problem into unconstrained problems has a **price**: conditioning.
6. **IN PRACTICE** (4.3.3): penalize, refuse or compare — the algorithm is the same.

Next (Ch. 5): these methods give local minima. What if there are several basins?

References and resources

References Full list: *Course bibliography*, at the end of the course PDF.

- Nocedal, J., Wright, S. J., *Numerical Optimization*, 2nd ed., Springer, 2006 — Sec. 17.2 (exact penalty, Thm. 17.3).
- Deb, K., *An efficient constraint handling method for genetic algorithms*, Computer Methods in Applied Mechanics and Engineering 186(2–4):311–338, 2000 — feasibility rule.
- Lewis, R. M., Torczon, V., *Pattern search methods for linearly constrained minimization*, SIAM J. Optim. 10(3):917–941, 2000.
- Audet, C., Dennis, J. E., Jr., *A progressive barrier for derivative-free nonlinear programming*, SIAM J. Optim. 20(1):445–472, 2009.
- Le Digabel, S., Wild, S. M., *A taxonomy of constraints in black-box simulation-based optimization*, Optimization and Engineering, 2023 (doi:10.1007/s11081-023-09839-3).

Resources

- Companion code: `ex04_3_3_constraint_handling`, `ExtremeBarrier`; `HookeJeeves` (3.2.5) with optional `menor`.
- MATLAB: `patternsearch` and `ga` accept constraints. NOMAD: progressive barrier.